

Table Creation (DDL Operations)

Student ID1: 23-52314-2	StudentID3: 23-50713-1
Name: FARHAN CHOWDHURY	Name: MD. ABDUR RAUF
StudentID2: 23-52361-2	StudentID4: 23-52586-2
Name: NAYEM SIDDIKI	Name: MD AL MURSALIN
CO4: Creating DML, DDL using Oracle and connection with ODBC/JDBC for existing JAVA application	
PO-e-2: Use modern engineering and IT tools for the prediction and modeling of complex computer science and engineering problem	Marks

Table 01:

☒ Automatic submission

```

CREATE TABLE Bus_Tickets (
    Ticket_No VARCHAR(10) PRIMARY KEY,
    Passenger_ID VARCHAR(10),
    Travel_Date DATE,
    Seat_No INT,
    Status VARCHAR(20),
    Bus_No VARCHAR(10),
    Route_No VARCHAR(10)
);
    
```

[result](#)
[explain](#)
[illustrate](#)
[Saved SQL](#)
[History](#)

The table is created.

[illegible]

Table 02:

☒ Automatic submission
 OK
 10

```

CREATE TABLE Buses (
  Bus_No VARCHAR(10) PRIMARY KEY,
  Seating_Capacity INT,
  Type VARCHAR(10),
  Driver_Name VARCHAR(100)
);
    
```

[result](#)
[explain](#)
[illustrate](#)
[Saved SQL](#)
[History](#)

The table is created.

DESCRIBE Buses;									
result explain illustrate Saved SQL History									
Object Type TABLE Object BUSES									
surface	List	Data Types	length	Accuracy	Decimal Places	Primary Key	Can be null	default value	Notes
BUSES	BUS_NO	VARCHAR2	10	-	-	1	-	-	-
	SEATING_CAPACITY	NUMBER	twenty two	-	0	-	✓	-	-
	TYPE	VARCHAR2	10	-	-	-	✓	-	-
	DRIVER_NAME	VARCHAR2	100	-	-	-	✓	-	-
									1 - 4

Table 03:

☒ Automatic submission OK 10  

```
CREATE TABLE Route (  
    Route_No VARCHAR(10) PRIMARY KEY,  
    Start_Point VARCHAR(100),  
    Destination VARCHAR(100),  
    Total_Distance VARCHAR(20)  
);
```

[result](#) [explain](#) [illustrate](#) [Saved SQL](#) [History](#)

The table is created.

DESCRIBE Route;									
result	explain	illustrate	Saved SQL	History					
Object Type TABLE Object ROUTE									
surface	List	Data Types	length	Accuracy	Decimal Places	Primary Key	Can be null	default value	Notes
ROUTE	ROUTE_NO	VARCHAR2	10	-	-	1	-	-	-
	START_POINT	VARCHAR2	100	-	-	-	✓	-	-
	DESTINATION	VARCHAR2	100	-	-	-	✓	-	-
	TOTAL_DISTANCE	VARCHAR2	20	-	-	-	✓	-	-
									1 - 4

Table 04:

☒ Automatic submission OK 10  

```
CREATE TABLE Passenger (  
    Passenger_ID VARCHAR(10) PRIMARY KEY,  
    Passenger_Name VARCHAR(100),  
    Contact_No VARCHAR(15)  
);
```

result explain illustrate Saved SQL History

The table is created.

DESCRIBE Passenger;									
result explain <u>illustrate</u> Saved SQL History									
Object Type TABLE Object PASSENGER									
surface	List	Data Types	length	Accuracy	Decimal Places	Primary Key	Can be null	default value	Notes
PASSENGER	PASSENGER_ID	VARCHAR2	10	-	-	1	-	-	-
	PASSENGER_NAME	VARCHAR2	100	-	-	-	✓	-	-
	CONTACT_NO	VARCHAR2	15	-	-	-	✓	-	-
									1 - 3

Inserted Values in the tables

Table: 01

-- Insert values into the Bus_Tickets table

```
INSERT INTO Bus_Tickets (Ticket_No, Passenger_ID, Travel_Date, Seat_No, Status, Bus_No, Route_No) VALUES ('T001', 'P001', TO_DATE('2024-09-18', 'YYYY-MM-DD'), 10, 'Confirmed', '001', '101');
```

```
INSERT INTO Bus_Tickets (Ticket_No, Passenger_ID, Travel_Date, Seat_No, Status, Bus_No, Route_No) VALUES ('T002', 'P002', TO_DATE('2024-09-18', 'YYYY-MM-DD'), 12, 'Booked', '001', '101');
```

```
INSERT INTO Bus_Tickets (Ticket_No, Passenger_ID, Travel_Date, Seat_No, Status, Bus_No, Route_No) VALUES ('T003', 'P003', TO_DATE('2024-09-19', 'YYYY-MM-DD'), 5, 'Confirmed', '002', '102');
```

```
INSERT INTO Bus_Tickets (Ticket_No, Passenger_ID, Travel_Date, Seat_No, Status, Bus_No, Route_No) VALUES ('T004', 'P004', TO_DATE('2024-09-19', 'YYYY-MM-DD'), 8, 'Canceled', '002', '102');
```

```
INSERT INTO Bus_Tickets (Ticket_No, Passenger_ID, Travel_Date, Seat_No, Status, Bus_No, Route_No) VALUES ('T005', 'P005', TO_DATE('2024-09-20', 'YYYY-MM-DD'), 15, 'Booked', '003', '103');
```

```
select * from Passenger;  
select * from Route;  
select * from Buses;  
select * from Bus_Tickets;
```

result [explain](#) [illustrate](#) [Saved SQL](#) [History](#)

TICKET_NO	PASSENGER_ID	TRAVEL_DATE	SEAT_NO	STATUS	BUS_NO	ROUTE_NO
T001	P001	2024-09-18	10	Confirmed	001	101
T005	P005	2024-09-20	15	Booked	003	103
T002	P002	2024-09-18	12	Booked	001	101
T003	P003	2024-09-19	5	Confirmed	002	102
T004	P004	2024-09-19	8	Canceled	002	102

5 rows returned after 0.02 seconds

[download](#)

Table: 02

-- Insert values into the Buses table

```
INSERT INTO Buses (Bus_No, Seating_Capacity, Type, Driver_Name) VALUES ('001', 50, 'AC', 'John Doe');
```

```
INSERT INTO Buses (Bus_No, Seating_Capacity, Type, Driver_Name) VALUES ('002', 40, 'Non-AC', 'Jane Smith');
```

```
INSERT INTO Buses (Bus_No, Seating_Capacity, Type, Driver_Name) VALUES ('003', 55, 'AC', 'Mike Green');
```

```
select * from Passenger;  
select * from Route;  
select * from Buses;  
select * from Bus_Tickets;
```

result [explain](#) [illustrate](#) [Saved SQL](#) [History](#)

BUS_NO	SEATING_CAPACITY	TYPE	DRIVER_NAME
001	50	AC	John Doe
002	40	Non-AC	Jane Smith
003	55	AC	Mike Green

3 rows returned after 0.01 seconds

[download](#)

Table: 03

-- Insert values into the Route table

```
INSERT INTO Route (Route_No, Start_Point, Destination, Total_Distance) VALUES ('101', 'City A', 'City B', '200 km');
```

```
INSERT INTO Route (Route_No, Start_Point, Destination, Total_Distance) VALUES ('102', 'City C', 'City D', '150 km');
```

```
INSERT INTO Route (Route_No, Start_Point, Destination, Total_Distance) VALUES ('103', 'City E', 'City F', '300 km');
```

<pre>select * from Passenger; select * from Route; select * from Buses; select * from Bus_Tickets;</pre>			
result explain illustrate Saved SQL History			
ROUTE_NO	START_POINT	DESTINATION	TOTAL_DISTANCE
101	City A	City B	200 km
102	City C	City D	150 km
103	City E	City F	300 km
3 rows returned in 0.00 seconds		download	

Table: 04

-- Insert values into the Passenger table

```
INSERT INTO Passenger (Passenger_ID, Passenger_Name, Contact_No) VALUES ('P001', 'Alice Green', '1234567890');
```

```
INSERT INTO Passenger (Passenger_ID, Passenger_Name, Contact_No) VALUES ('P002', 'Bob Brown', '0987654321');
```

```
INSERT INTO Passenger (Passenger_ID, Passenger_Name, Contact_No) VALUES ('P003', 'Carol White', '1122334455');
```

```
INSERT INTO Passenger (Passenger_ID, Passenger_Name, Contact_No) VALUES ('P004', 'David Black', '6677889900');
```

```
INSERT INTO Passenger (Passenger_ID, Passenger_Name, Contact_No) VALUES ('P005', 'Eve Brown', '2233445566');
```

<pre>select * from Passenger; select * from Route; select * from Buses; select * from Bus_Tickets;</pre>						
result explain illustrate Saved SQL History						
TICKET_NO	PASSENGER_ID	TRAVEL_DATE	SEAT_NO	STATUS	BUS_NO	ROUTE_NO
T001	P001	2024-09-18	10	Confirmed	001	101
T005	P005	2024-09-20	15	Booked	003	103
T002	P002	2024-09-18	12	Booked	001	101
T003	P003	2024-09-19	5	Confirmed	002	102
T004	P004	2024-09-19	8	Canceled	002	102
5 rows returned in 0.00 seconds download						

Query Test in DB

There will be one simple query. One with Group/ Aggregate function, 2 single row Subquery, 2 Multiple Row subquery, 4 different types of joining.

a) Simple Query

Question: Print details from ticket table where status is confirmed.

```
= SELECT TICKET_NO, TRAVEL_DATE, SEAT_NO FROM BUS_TICKETS WHERE STATUS = 'CONFIRMED';
```

TICKET_NO	TRAVEL_DATE	SEAT_NO
T001	2024-09-18	10
T003	2024-09-19	5

2 rows returned after 0.01 seconds [download](#)

b) Query with a Single Row Function

Question: Find the name of the passenger who has the longest contact number.

```
= SELECT PASSENGER_NAME FROM PASSENGER WHERE LENGTH(CONTACT_NO) = (SELECT MAX(LENGTH(CONTACT_NO)) FROM PASSENGER);
```

PASSENGER_NAME
Alice Green
Bob Brown
Carol White
David Black
Eve Brown

5 rows returned in 0.00 seconds

C) Query with a Multiple Row Function / Aggregate Function

Question: Calculate the total number of booked tickets for each travel date.

```
= SELECT TRAVEL_DATE, COUNT(*) AS TOTAL_BOOKED FROM BUS_TICKETS WHERE  
STATUS = 'BOOKED' GROUP BY TRAVEL_DATE;GROUP BY TRAVEL_DATE;
```

TRAVEL_DATE	TOTAL_BOOKED
2024-09-18	1
2024-09-20	1

2 rows returned after 0.00 seconds [download](#)

d) 2 Single Row Subqueries and 2 Multiple Row Subqueries

Single Row Subqueries:

1. **Question:** Find the name of the passenger with Ticket_No 'T001'.

```
= SELECT PASSENGER_NAME FROM PASSENGER WHERE PASSENGER_ID =  
(SELECT PASSENGER_ID FROM BUS_TICKETS WHERE TICKET_NO = 'T001');
```

PASSENGER_NAME
Alice Green

1 row returned after 0.07 seconds

2. **Question:** Get the seating capacity of the bus with Bus_No '002'.

```
= SELECT SEATING_CAPACITY FROM BUSES WHERE BUS_NO = (SELECT BUS_NO  
FROM BUS_TICKETS WHERE TICKET_NO = 'T004');
```

SEATING_CAPACITY
40

1 row returned after 0.01 seconds

Multiple Row Subqueries:

1. **Question:** Find all passengers who have booked tickets on the same travel date as '2024-09-18'.

= SELECT PASSENGER_NAME FROM PASSENGER WHERE PASSENGER_ID IN (SELECT PASSENGER_ID FROM BUS_TICKETS WHERE TRAVEL_DATE IN (SELECT TRAVEL_DATE FROM BUS_TICKETS WHERE TRAVEL_DATE = '2024-09-18'));

PASSENGER_NAME
Alice Green
Bob Brown

2 rows returned after 0.02 seconds

2. **Question:** List all ticket numbers for passengers who have a contact number starting with '1'.

= SELECT TICKET_NO FROM BUS_TICKETS WHERE PASSENGER_ID IN (SELECT PASSENGER_ID FROM PASSENGER WHERE CONTACT_NO LIKE '1%');

TICKET_NO
T001
T003

2 rows returned after 0.01 seconds

e) 4 kinds of joining

1. Inner Join:

Question: Get a list of all tickets along with passenger names.

```
= SELECT BT.TICKET_NO, P.PASSENGER_NAME FROM BUS_TICKETS BT INNER  
JOIN PASSENGER P ON BT.PASSENGER_ID = P.PASSENGER_ID;
```

TICKET_NO	PASSENGER_NAME
T001	Alice Green
T002	Bob Brown
T003	Carol White
T004	David Black
T005	Eve Brown

5 rows returned after 0.01 seconds [download](#)

2. Left Join:

Question: List all buses and their tickets (if any), showing buses even if they have no tickets.

```
= SELECT B.BUS_NO, BT.TICKET_NO FROM BUSES B LEFT JOIN BUS_TICKETS BT  
ON B.BUS_NO = BT.BUS_NO;
```

BUS_NO	TICKET_NO
001	T001
003	T005
001	T002
002	T003
002	T004

5 rows returned after 0.01 seconds

3. Right Join:

Question: Get a list of all tickets and the buses they are assigned to, showing tickets even if there are no associated buses.

```
= SELECT BT.TICKET_NO, B.BUS_NO FROM BUS_TICKETS BT RIGHT JOIN BUSES  
B ON BT.BUS_NO = B.BUS_NO;
```

TICKET_NO	BUS_NO
T001	001
T005	003
T002	001
T003	002
T004	002

5 rows returned after 0.01 seconds

4. Full Outer Join:

Question: Retrieve all passengers and their booked tickets, if any, showing all passengers and tickets.

```
= SELECT P.PASSENGER_NAME, BT.TICKET_NO FROM PASSENGER P FULL  
OUTER JOIN BUS_TICKETS BT ON P.PASSENGER_ID = BT.PASSENGER_ID;
```

PASSENGER_NAME	TICKET_NO
Alice Green	T001
Eve Brown	T005
Bob Brown	T002
Carol White	T003
David Black	T004

5 rows returned after 0.01 seconds [download](#)

1 simple view

Question: Question: Create a view called Simple_Bus_Ticket_Info that displays ticket numbers, passenger names, and travel dates for all tickets.

= **CREATE VIEW** Simple_View **AS**

SELECT

BT.Ticket_No,P.Passenger_Name,BT.Travel_Date

FROM

Bus_Tickets BT

JOIN

Passenger P ON BT.Passenger_ID = P.Passenger_ID;

select * from Simple_View; DESCRIBE Simple_View;										
result explain illustrate Saved SQL History										
Object Type VIEW Object SIMPLE_VIEW										
surface	List	Data Types	length	Accuracy	Decimal Places	Primary Key	Can be null	default value	Notes	
SIMPLE_VIEW	TICKET_NO	VARCHAR2	10	-	-	-	-	-	-	-
	PASSENGER_NAME	VARCHAR2	100	-	-	-	✓	-	-	-
	TRAVEL_DATE	DATE	7	-	-	-	✓	-	-	-
										1 - 3

select * from Simple_View;		
result explain illustrate Saved SQL History		
TICKET_NO	PASSENGER_NAME	TRAVEL_DATE
T001	Alice Green	2024-09-18
T002	Bob Brown	2024-09-18
T003	Carol White	2024-09-19
T004	David Black	2024-09-19
T005	Eve Brown	2024-09-20
5 rows returned after 0.01 seconds		download

1 complex view

Question: Create a view called Complex_View that includes ticket numbers, passenger names, travel dates, seat numbers, bus details, and route information, ensuring that all records are included from all relevant tables.

= **CREATE VIEW** Complex_View **AS**

SELECT

BT.Ticket_No,P.Passenger_Name,BT.Travel_Date,BT.Seat_No,B.Bus_No,B.Driver_Name,R.St
art_Point,R.Destination,R.Total_Distance

FROM

Bus_Tickets BT

INNER JOIN

Passenger P ON BT.Passenger_ID = P.Passenger_ID

LEFT JOIN

Buses B ON BT.Bus_No = B.Bus_No

FULL OUTER JOIN

Route R ON BT.Route_No = R.Route_No

WHERE

BT.Status = 'Confirmed' OR BT.Status IS **NULL**;

DESCRIBE Complex_View;

resultexplainillustrateSaved SQLHistory

Object Type	VIEW	Object	COMPLEX_VIEW							
surface	List	Data Types	length	Accuracy	Decimal Places	Primary Key	Can be null	default value	Notes	
COMPLEX_VIEW	TICKET_NO	VARCHAR2	10	-	-	-	✓	-	-	
	PASSENGER_NAME	VARCHAR2	100	-	-	-	✓	-	-	
	TRAVEL_DATE	DATE	7	-	-	-	✓	-	-	
	SEAT_NO	NUMBER	twenty two	-	0	-	✓	-	-	
	BUS_NO	VARCHAR2	10	-	-	-	✓	-	-	
	DRIVER_NAME	VARCHAR2	100	-	-	-	✓	-	-	
	START_POINT	VARCHAR2	100	-	-	-	✓	-	-	
	DESTINATION	VARCHAR2	100	-	-	-	✓	-	-	
	TOTAL_DISTANCE	VARCHAR2	20	-	-	-	✓	-	-	

1 - 9

select * from Detailed_Confirmed_Bus_Tickets;

resultexplainillustrateSaved SQLHistory

TICKET_NO	PASSENGER_NAME	TRAVEL_DATE	SEAT_NO	BUS_NO	DRIVER_NAME	START_POINT	DESTINATION	TOTAL_DISTANCE
T001	Alice Green	2024-09-18	10	001	John Doe	City A	City B	200 km
T003	Carol White	2024-09-19	5	002	Jane Smith	City C	City D	150 km

2 rows returned after 0.02 secondsdownload